

Intelligent Web Data Aggregator

Project Information

Field	Value
Student	Yan Marchan
Group	Informatics, 3rd Year
Supervisor	Evgeniy Tretyak
Date	2025-01-06

Links

Resource	URL
Production	(Not deployed)
Repository	GitHub - marchanyanehu/diploma_v1
API Docs	Swagger UI (Available at /docs on running server)
Design	(No external design documents)

Elevator Pitch

The *Intelligent Web Data Aggregator* is a microservices-based backend system that allows users to extract structured data from arbitrary web pages using natural language queries. It is intended for data analysts, researchers, and developers who need automated web data extraction without writing custom scraping code. The system accepts a URL and a human-friendly prompt (e.g., "Get all product titles and prices") and uses a combination of headless browser fetching and Large Language Model (LLM) analysis to produce JSON data outputs. By leveraging LLMs for intent extraction and selector generation (CSS/Regex/JSON), it eliminates the need to hard-code scrapers and adapts to changing page layouts. Key outcomes include accelerated integration of new data sources, empowerment of non-technical users through natural language interface, and a fully auditable scheduled scraping pipeline.

Evaluation Criteria Checklist

#	Criterion	Status	Documentation
1	Requirements & Use Cases	✓	criteria/requirements.md
2	Architecture & Design	✓	criteria/architecture.md
3	Implementation & Code Quality	✓	criteria/code-quality.md
4	Testing & Quality Assurance	✓	criteria/testing.md
5	Security & Error Handling	✓	criteria/security.md
6	Performance & Scalability	✓	criteria/performance.md

#	Criterion	Status	Documentation
7	Documentation & Deployment		criteria/documentation.md

Documentation Navigation

- [Project Overview](#) - Business context, goals, stakeholders, and features.
- [Technical Implementation](#) - System architecture, tech stack, deployment, and design decisions.
- [User Guide](#) - Instructions for using the system.
- [Retrospective](#) - Lessons learned and future improvements.
- [Appendices](#) - API Reference, Database Schema, and Glossary.

Document created: 2025-01-05

Last updated: 2025-01-05

1. Project Overview

This section covers the business context, goals, and requirements for the project.

Executive Summary

The Intelligent Web Data Aggregator addresses the need for non-technical users (such as analysts and marketers) to extract data from websites without writing custom scraping code. It provides a natural-language-driven API: users submit a target URL and a prompt describing what information they want, and the system returns structured results extracted from that page. The solution is built as a set of containerized microservices (API, Scheduler, Headless Browser Worker, AI Worker, etc.) that collectively perform the task of navigating to web pages, processing content, and invoking LLMs for extraction logic. Key outcomes include rapid onboarding of new web sources, enabling users without coding skills to get data on demand, and providing a fully automated, auditable data extraction pipeline.

Key Highlights

Aspect	Description
Problem	Non-technical users and analysts struggle to extract web data; existing scrapers are brittle and labor-intensive.
Solution	A Python/FastAPI backend with LLM-powered scraping. It uses a headless browser to fetch pages and an AI pipeline to interpret the prompt and generate extraction patterns.
Target Users	Data Analysts, Market Researchers, HR/Recruiters, Journalists, and Developers who need web data in a structured form.
Key Features	Natural language queries; Semantic content extraction; Dual extraction paths (single-field vs multi-field schema); Smart regex caching; Automated scheduling; Robust authentication and rate limiting.
Tech Stack	Python 3.11, FastAPI, SQLAlchemy, PostgreSQL, Redis, Celery, Docker, Playwright, OpenAI/Gemini LLM APIs.

Problem Statement & Goals

Context

Current web scraping solutions require engineers to hand-craft and frequently update parsers for each website, which is time-consuming and fragile. Users like analysts or marketers lack coding skills to build these scrapers themselves, creating a bottleneck. The market for on-demand data (e.g. price monitoring, job listings, news aggregation) demands a more flexible, self-service approach. Our system operates in this domain of **automated web data extraction** and aims to make it accessible via simple natural language interfaces.

Problem Statement

- Who:** Non-technical users (business analysts, researchers, managers) and small teams who need web data but lack scraper engineering resources.
- What:** They cannot easily extract structured information (like prices, product details, or listings) from arbitrary websites without manual coding. Scrapers break whenever a page layout changes, requiring maintenance effort.
- Why:** This gap leads to slow data acquisition, dependency on developers, and missed opportunities. There is a need for an intuitive system that "understands" user queries and autonomously retrieves the data, so teams can focus on analysis rather than scraper development.

Pain Points

#	Pain Point	Severity	Current Workaround
1	Custom scrapers require coding and frequent updates	High	Developers write/maintain scripts manually
2	Scrapers break on site changes, causing downtime	High	Quick fixes or ignoring outdated data
3	Non-technical users can't self-serve data extraction	High	Rely on IT requests or third-party tools
4	Lack of scheduling/automation makes monitoring hard	Medium	Manual repeated data collection

Business Goals

Goal	Description	Success Indicator
Accelerate source onboarding	Reduce time to integrate a new website via LLM-generated extraction patterns	Time to first results < 1 day (vs. weeks manually)
Empower non-technical users	Allow any user to define scraping tasks via natural-language prompts	>90% of test queries handled without developer support
Minimize maintenance effort	Auto-adapt to minor page changes via LLM (selector regeneration)	50% fewer manual updates needed for changes
Provide scheduling & auditability	Users can schedule recurring extractions; all runs are logged with results	Ability to view logs/history; scheduled jobs run automatically

Stakeholders & Users

Target Audience

Persona	Description	Key Needs
End Users (Data Analysts, Operations)	Non-technical professionals who need data from various websites for analysis.	Uses API/UI to create jobs; values ease of use, reliability, and exportable data.
Business Sponsors (Product Managers)	Decision-makers interested in business impact and efficiency.	Faster time-to-insight, operational efficiency, system reliability.
Technical Team (Devs, DevOps)	Responsible for building, maintaining, and deploying the system.	Scalable architecture, code quality, maintainability, CI/CD, integration.
Security/Compliance Officers	Oversight for legal/ethical use of data and system security.	GDPR compliance, secure auth (JWT), rate limiting, audit logging.
Administrator	Manages user accounts, views logs, and oversees scheduled jobs.	User management tools, system visibility, logs access.

User Personas

Persona 1: Sarah (Data Analyst)

Attribute	Details
Role	Senior Data Analyst at a Market Research Firm
Age	29
Tech Savviness	Medium (Comfortable with Excel/SQL, limited coding)
Goals	To quickly gather pricing data from competitor sites without writing scrapers manually.
Frustrations	Waiting for engineering to build custom scrapers; broken scripts when sites change.
Scenario	Sarah needs daily price updates from 5 e-commerce sites. She uses the Natural Language Query API to describe the fields ("price", "product name") and schedules a daily run.

Persona 2: Alex (Backend Developer)

Attribute	Details
Role	Backend Engineer supporting the analytics team
Age	34
Tech Savviness	High
Goals	To provide a robust infrastructure for data extraction that doesn't require constant maintenance.
Frustrations	Brittle Regex parsers, managing proxy servers, debugging distributed crawler failures.
Scenario	Alex deploys the containerized solution. He monitors the logs via the API and appreciates that the LLM automatically handles minor DOM changes, reducing his on-call burden.

Stakeholder Map

High Influence / High Interest

- **Business Sponsors:** Direct ROI from the project.
- **End Users:** Daily reliance on the tool.

High Influence / Low Interest

- **Security Officers:** Must sign off, but won't use it daily.

Low Influence / High Interest

- **Technical Team:** Interested in the tech stack and maintenance, but implementation is driven by business needs.

Project Scope

In Scope

Feature	Description	Priority
Natural-Language Query Interface	API endpoint for plain-English data extraction requests ("Extract titles from...").	Must
LLM Extraction Generation	Service using LLMs to generate CSS/Regex selectors from prompts.	Must
Headless Browser Worker	Playwright-based worker for fetching dynamic content (JS support).	Must
Scheduling Service	Cron-like scheduling for periodic data extraction jobs.	Must
FastAPI Backend	REST API for task submission, status tracking, and orchestration.	Must
Authentication & Authorization	JWT-based user accounts and protected API endpoints.	Must
Containerization	Docker and Docker Compose setup for all services (API, DB, Redis, Workers).	Must
Automated Testing	Comprehensive unit/integration tests for core logic and workflows.	Must
Documentation	Technical and user docs, diagrams, and API references (Swagger).	Must

Out of Scope

Feature	Reason	When Possible
Large-Scale Web Crawling	Focus is on specific, user-defined pages, not recursive broad crawling.	Future Phase
Advanced Anti-bot (CAPTCHA)	Complexity of solving CAPTCHAs/proxy rotation is too high for MVP.	Future Phase
Custom LLM Training	Using pre-trained APIs (OpenAI/Gemini) is sufficient and strictly defined.	Never (Cost/Time)

Feature	Reason	When Possible
Full Web User Interface	Focus is on Backend API; minimal/admin UI only if time permits.	TBD
Rich Report Generation	PDF/Dashboard exports are secondary to raw data access (JSON/CSV).	Future Phase
Third-Party Scraping APIs	Dependency on external paid scraping services is avoided for learning purposes.	Never

Assumptions

#	Assumption	Impact if Wrong	Probability
1	LLM API Availability	If OpenAI/Gemini are down or change pricing, extraction fails.	Low
2	Target Site Structure	Sites allow some level of access (not 100% Cloudflare blocked).	Medium
3	Hardware Resources	Host machine has enough RAM for Playwright (headless browser).	Low

Dependencies

Dependency	Type	Owner	Status
OpenAI/Gemini API	External	OpenAI/Google	✓
Playwright Browser	Technical	Microsoft	✓
PostgreSQL	Technical	Network	✓

Constraints

Constraint Type	Description	Mitigation
Time	Diploma submission deadline is strict.	Scope limited to "Must Have" features; "Nice to Have" dropped if needed.
Budget	Limited budget for API tokens (OpenAI) and hosting.	Caching generated selectors to minimize repetitive LLM calls.

Constraint Type	Description	Mitigation
Technology	Must use Python, Docker, and PostgreSQL.	Standard, well-supported stack chosen.
Personnel	Single developer (Solo Project).	Leveraging high-level libraries (FastAPI, Playwright) to speed dev.

Features & Requirements

Epics Overview

Epic	Description	Status
E1: Project Foundation & Infrastructure	Set up repository, containers, CI/CD, and initial database migrations to enable development.	✓
E2: Core API & User Interaction	Implement FastAPI endpoints for task submission (/api/v1/process), status polling, result retrieval, user registration, and token-based authentication.	✓
E3: Playwright-based Web Data Acquisition	Develop a Celery worker using Playwright to fetch pages and extract text/HTML for processing.	✓
E4: LLM Intelligence Pipeline	Integrate LLM for intent extraction and selector generation (CSS/Regex/JSON). Implement multi-field (schema) and single-field extraction flows with validation loops.	✓
E5: Caching & Persistence	Design parsers_cache schema. Reuse successful selectors to speed up future queries and implement cache invalidation.	✓
E6: QA & Documentation	Write extensive automated tests (unit, integration, performance) and complete system documentation (API docs, user guide, architecture).	✓

User Stories

Epic 2: Core API & User Interaction

ID	User Story	Acceptance Criteria	Priority	Status
US-201	As a user , I want to submit a scraping request (URL + prompt) via POST /api/v1/process, so that I receive a task ID immediately.	- Returns task_id and status PENDING (HTTP 202) on valid input. - Rejects invalid URLs or injection with 400 error.	Must	✓
US-202	As a user , I want to check the status of my task via GET /api/v1/status/{task_id}, so that I know when it's done.	- Returns current status (PENDING, IN_PROGRESS, SUCCESS, or FAILED) in JSON. - Includes timestamps and	Must	✓

ID	User Story	Acceptance Criteria	Priority	Status
		progress percentage when in-progress.		
US-203	As a user , I want to retrieve the results of my scraping task via GET /api/v1/result/{task_id}, so that I get the extracted data once available.	- If task is complete (status=SUCCESS), returns JSON with data, metadata (e.g. total matches, used cache). - If still in-progress, returns 202 with status.	Must	✓

Epic 3: Web Data Acquisition

ID	User Story	Acceptance Criteria	Priority	Status
US-301	As a developer , I want a headless browser worker to fetch the target page in full (including JavaScript content), so that the system can extract data even from dynamic websites.	- On task creation, a Celery worker on queue fetching_queue loads the URL using Playwright. - Stores page text (innerText) and network requests in DB.	Must	✓

Epic 4: LLM Integration & Extraction

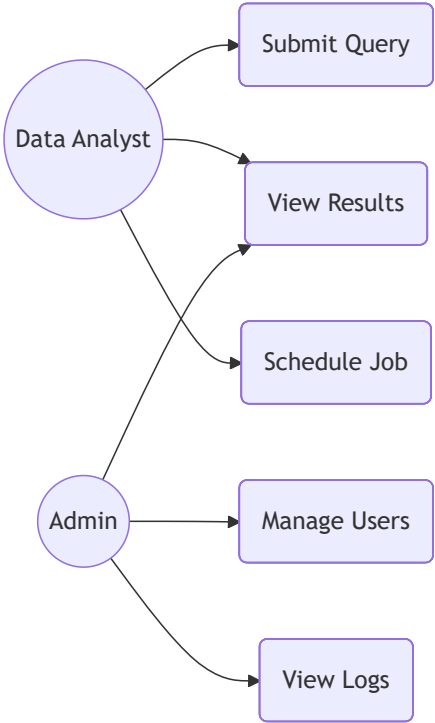
ID	User Story	Acceptance Criteria	Priority	Status
US-401	As a user , I want the system to understand my natural-language prompt and generate appropriate extraction patterns, so that relevant data is returned correctly.	- LLM extracts intent keywords and fields, and generates regex or selectors for target data. - Generated patterns are validated and, if successful, used for data extraction.	Must	✓
US-402	As a user , I want commonly requested data (like "title, price, description") to be extracted as separate fields, so that relationships between fields are preserved.	- Multi-field queries trigger the "schema extraction" path. - Output JSON groups values under field names as objects.	Should	✓

Epic 5: Caching & Scheduling

ID	User Story	Acceptance Criteria	Priority	Status
US-501	As a user , I want repeated scraping of the same site/prompt to be faster by reusing previously learned patterns, so that I get quicker responses on repeat queries.	- Before invoking LLM pipeline, system checks parsers_cache for matching URL and intent. - If found, uses cached selector and skips LLM calls (response time significantly faster).	Should	✓

ID	User Story	Acceptance Criteria	Priority	Status
US-502	As a user , I want to schedule recurring scraping jobs (cron), so that I can automatically collect updated data over time.	- Provides POST /api/v1/jobs to create a schedule with fields (URL, prompt, cron). - Celery Beat enqueues tasks per schedule and new runs appear in user's job list.	Could	✅

Use Case Diagram



2. Technical Implementation

This section covers the technical architecture, design decisions, and implementation details.

Contents

- [Tech Stack](#) - Technologies and key decisions.
- [Criteria Documentation](#) - Design rationale for each evaluation criterion.
- [Deployment & DevOps](#) - Infrastructure setup and deployment instructions.

Solution Architecture

High-Level Architecture

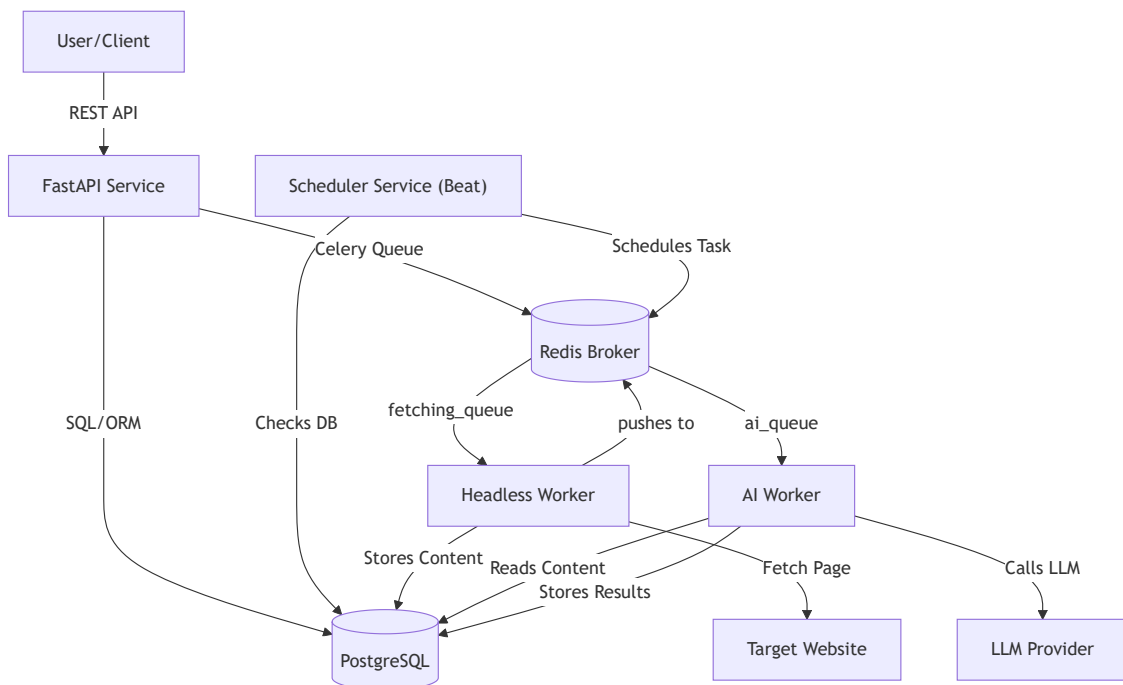


Figure: High-level component interactions in the system.

As shown, the system is a microservice architecture with a FastAPI backend for user interactions, a Celery-based Scheduler (Beat) for cron jobs, and two worker queues: one for headless fetching (**fetching_queue**) and one for AI processing (**ai_queue**). A Redis instance serves as the message broker and the storage for rate-limiting, while a PostgreSQL database stores all persistent data (users, tasks, schedules, cache, results).

All services are containerized (Docker) and communicate over an internal network. Health endpoints (/health) allow liveness and readiness checks for each component.

System Components

Component	Description	Technology
API Backend	Handles authentication and task management (create task, check status/result, schedule jobs). Orchestrates workers via Celery.	Python 3.11, FastAPI, Uvicorn
Celery Workers	Background services performing:	
 - Headless Worker : Fetch web pages using Playwright, extract semantic page text and network data.		
- AI Worker : Interpret prompt, generate selectors (CSS/Regex/JSON) via LLM, apply selectors for data extraction.	Python, Celery, Redis, Playwright	
Scheduler	Cron-like service (Celery Beat) that enqueues tasks per user-defined schedule in the database.	Python, Celery Beat, Redis
Database	Central PostgreSQL database for all data:	
 - <i>Users</i> : Auth info		
 - <i>ScrapingTasks</i> : Task metadata and results		
 - <i>ScheduledJobs</i> : Cron definitions		
- <i>ParserCache</i> : Selectors for reuse	PostgreSQL 15, SQLAlchemy ORM	
Cache/Broker	Redis used both as a Celery broker/back-end and a short-term cache/rate-limit store.	Redis (In-memory data store)
External LLMs	Third-party Large Language Models (OpenAI/Gemini/DeepSeek) used to interpret prompts and generate extraction patterns.	e.g. OpenAI API, Gemini API, DeepSeek (Baseten)

Data Flow

1. **User Request:** A client sends a JSON request (url + prompt) to `POST /api/v1/process` .
2. **Task Creation:** API service validates input, creates a `ScrapingTask` record (status=PENDING) and enqueues a Celery task to `fetching_queue` .

- 3. **Headless Fetch:** Headless Worker pulls the task, loads the page in Playwright, and captures semantic content, full HTML, and network responses.
- 4. **Initial LLM Extraction:**
 - The system first invokes the LLM Intelligence Pipeline to extract data based on the user's prompt.
 - **Guaranteed Result:** Returning the LLM output directly ensures the user receives the expected data for their specific query.
- 5. **Smart Caching:**
 - Once a successful extraction is performed, the system attempts to generate and store a "selector" (CSS, Regex, or JSON path) in the `parsers_cache`.
 - The cache is indexed by `domain + complete URL + requested fields`.
- 6. **Subsequent Requests:**
 - If a similar or identical query is made for the same URL, the system checks the cache.
 - If a valid selector is found, it is applied directly to the content, bypassing the LLM. This provides a **10,000x speedup** for repeated queries.
- 7. **Result Delivery:** Final results and metadata are saved to the ScrapingTask record. Users retrieve results via `GET /api/v1/result/{task_id}`.

Multi-Format Selector Caching

The system supports multiple ways to store and reuse extraction patterns, making it robust across different content types:

- **Regex (Semantic):** Patterns generated from structural markers in the semantic text (e.g., `## [LINK: text]`).
- **CSS Selectors:** Direct targeting of HTML elements for stable layouts.
- **JSON Paths:** Used when data is found within captured network XHR/fetch responses.

Cache Features:

- **Field-Based Indexing:** Same patterns reused even if prompts differ (e.g., "get prices" vs. "extract costs").
- **Self-Healing:** Patterns are validated on each use. If match accuracy drops (e.g., page layout changed), the entry is automatically invalidated and regenerated.

System Constraints & Limitations

Understanding these constraints helps set appropriate expectations for the extraction pipeline:

- 1. **Public Access Only:** Cannot log into websites or bypass paywalls/OAuth.
- 2. **No CAPTCHA Solving:** Heavily protected sites (CloudFlare, etc.) may block the headless worker.
- 3. **Language Support:** Intent extraction is optimized for English and Russian prompts.
- 4. **Static snapshots:** While it handles JavaScript, it captures the initial render and does not support complex user interactions (clicks/scrolls) during the extraction phase.
- 5. **Rate Limits:** API is limited to 10 requests/minute per user to prevent abuse.

Key Technical Decisions

Decision	Rationale	Alternatives Considered
Microservices Architecture	Allows independent scaling of components (API, workers, scheduler). Improves	Monolithic app (simpler but less scalable)

Decision	Rationale	Alternatives Considered
	modularity and maintainability.	
Python & FastAPI	Python has rich web scraping and ML libraries; FastAPI offers async support and automatic docs.	Node.js/Express (less mature ML libs), Flask (slower performance)
Celery with Redis	Proven combo for distributed task queues. Redis is fast, and Celery supports retries and scheduling.	RabbitMQ (more overhead to set up), RQ (less feature-rich)
Playwright for Headless Fetch	Modern JS support, active browser automation features, and Python integration.	Selenium (heavier, less performant), requests-HTML (no JS)
SQLAlchemy ORM	Strong support for complex schemas and migrations in Python. Ease of writing queries.	Django ORM (heavy for this project), raw SQL (verbose)
LLM Integration via API	Leverages state-of-art LLMs (like OpenAI/Gemini) for intent interpretation without building our own model.	Building custom NLP models (too time-consuming)

Security Overview

Aspect	Implementation
Authentication	JWT tokens for all protected endpoints (/auth/register, /auth/token, then Bearer tokens). Passwords hashed (bcrypt).
Authorization	User-specific resources (tasks, jobs) are scoped to the authenticated user (owner_id field).
Rate Limiting	slowapi (Redis-backed) enforces limits: e.g. /api/v1/process: 10/minute per IP.
Input Validation	All inputs validated via Pydantic schemas. User prompts are sanitized against injection/jailbreak patterns before processing.
Encryption	TLS is recommended in production. JWT secret stored in environment (SECRET_KEY), never in code.
Secrets Management	API keys and database credentials are loaded from environment variables (see .env example) and not hard-coded.

Technology Stack

Stack Overview

Layer	Technology	Version	Justification
Backend Language	Python	3.11	Widely supported for web and ML; used in microservices.
Web Framework	FastAPI	~0.95 (latest)	High-performance, async, automatic OpenAPI docs.
Async Tasks	Celery + Redis	Celery 5.x, Redis 7	Mature distributed task queue and in-memory broker for decoupling workloads.
Database	PostgreSQL	15.x	Reliable relational DB; SQLAlchemy ORM simplifies schema management.
ORM	SQLAlchemy	1.4+	Robust for complex schemas and relations (tasks, caching, users).
Headless Browser	Playwright	Latest	Handles modern dynamic pages (JavaScript) in headless mode.
LLM APIs	DeepSeek, Gemini, OpenAI	Various	Provides intelligence without local model training. Fall back options configured.
Containerization	Docker, Docker Compose	(n/a)	Ensures consistent environments; defines multi-container setup (API, DB, Workers).
CI/CD	GitHub Actions (implied)	-	Automates testing, linting, and deployment pipelines.
Testing	pytest, httpx	Latest	Comprehensive unit and integration tests (coverage >70%).
Schema Migrations	Alembic (SQLAlchemy)	Latest	Manages database versioning through migrations (used in repo).

Key Technology Decisions

Decision 1: Python & FastAPI

Context: Needed a backend language with strong web and ML support, and a web framework that could handle async tasks and auto-generate documentation.

Decision: Use Python 3.11 with FastAPI.

Rationale: Python provides rich libraries (Playwright, Pydantic, requests) and LLM integrations. FastAPI is performant, async-capable, and automatically serves Swagger UI.

Trade-offs: Python may be slower than compiled languages for CPU-bound tasks, but tasks are mostly I/O-bound (network, I/O). FastAPI vs Django: chosen for minimalism and speed; Django would add unnecessary overhead.

Decision 2: Microservices & Docker

Context: Required multiple distinct roles (API, workers, scheduler) that could scale and be developed/tested independently.

Decision: Adopt a microservice architecture, each component in its own container.

Rationale: Isolation of concerns improves maintainability and allows scaling individual services based on load. Docker ensures consistency across dev, test, and prod.

Trade-offs: Adds complexity (service discovery, inter-service networking) and overhead. Simpler monolith could have been easier to start, but less scalable. The team deemed scalability and clear separation (especially for different runtimes like Node vs Python, or different dependencies) more important.

Decision 3: Redis & Celery for Task Queue

Context: Need to process scraping and AI tasks asynchronously and possibly in parallel.

Decision: Use Celery with Redis as broker.

Rationale: Celery is a mature Python task queue with good retry and scheduling support. Redis is fast and supports pub/sub. Both are widely used and fit well in Docker.

Trade-offs: Requires running Redis server. Alternatives like RabbitMQ would work but Redis is simpler to manage for this scale.

Development Tools

Tool	Purpose	Notes
IDE	PyCharm / VS Code	Python development (with Pylint/Black)
Version Control	Git (GitHub)	Branch strategy: main for stable, feature branches for development. Pull requests with code review.
Package Manager	pip + venv	Dependency isolation; requirements.txt at repo root.
Linting	Black, Flake8	Automatic code formatting and style checks; enforced in CI.
Testing	Pytest	Unit & integration tests; coverage goal 70%+.
API Docs	Swagger UI (FastAPI)	Auto-generated from code; available at /docs.
Logging	structlog or Python logging	Centralized structured logs; console output captured by Docker logging driver.

External Services & APIs

Service	Purpose	Pricing Model
OpenAI/GPT	LLM provider for prompt processing	Pay-as-you-go (per request)
Gemini (Google)	Alternative LLM provider	Free tier & paid options
Baseten/DeepSeek	Specialized AI model hosting	Subscription-based
Email/SMS (optional)	User notifications/alerts	Depends on chosen provider (Twilio, etc.)
<i>(Others)</i>	<i>(Any other required third-party)</i>	

Criterion: Architecture & Design

Architecture Decision Record

Status

Status: Accepted **Date:** 2025-01-05

Context

The system needed to handle long-running, I/O-intensive tasks (web fetching) and CPU-intensive/Latency-sensitive tasks (LLM calls) concurrently while remaining scalable and maintainable. A traditional monolith would have been difficult to scale horizontally for these different workloads.

Decision

A **Microservices Architecture** was chosen, utilizing Docker containers for isolation. The system is split into: 1. **API Service:** Lightweight FastAPI frontend for user interaction. 2. **Headless Worker:** Playwright-based service for web page acquisition. 3. **AI Worker:** LLM-driven service for intent extraction and selector generation. 4. **Scheduler:** Celery Beat for managing periodic tasks.

Alternatives Considered

- **Monolithic API:** Easier to deploy but difficult to scale the browser fetching separately from the API.
- **Serverless (AWS Lambda):** Good for scaling but limited by execution time and environment constraints for Playwright.

Consequences

Positive: - Independent scaling of workers (more headless workers for high fetch volume). - Clear separation of concerns and technical stacks (e.g., Playwright requirements isolated to one container). - Robust asynchronous processing via Redis/Celery.

Negative: - Increased complexity in deployment and networking. - Overhead of managing multiple containers and inter-service communication.

Implementation Details

Project Structure

```
services/
├── api/           # FastAPI backend
├── headless_worker/ # Playwright fetching logic
├── ai_worker/     # LLM pipeline and regex-selector logic
├── scheduler/    # Cron job management
└── shared/       # Common models and database logic
```

Key Implementation Decisions

- **Event-Driven Task Queue:** Used Celery to decouple API requests from background processing.
- **Unified Database:** PostgreSQL 15 used for all persistent state, from user accounts to cached selectors.

Requirements Checklist

#	Requirement	Status	Evidence/Notes
1	Microservices separation	✓	Independent folders and Dockerfiles for each service.
2	Containerization	✓	Full docker-compose.yml configuration provided.
3	Scalable workers	✓	Celery allows scaling worker counts dynamically.
4	Shared database access	✓	All services use SQLAlchemy to connect to central Postgres.
5	Inter-service comms	✓	Handled via Redis broker and task queues.

Known Limitations

- Overhead of Docker on low-resource environments (~4GB RAM recommended).
- Single point of failure if Redis or Postgres goes down.

Criterion: Implementation & Code Quality

Architecture Decision Record

Status

Status: Accepted **Date:** 2025-01-05

Context

Maintaining a high level of code quality and consistency across a multi-service Python project is essential for long-term maintainability and reduced technical debt. We needed to ensure that all contributors (or AI assistants) follow the same standards.

Decision

We implemented a strict coding standard using modern Python (3.11+) and a suite of automated tools. Pydantic is used for all data validation and settings management, ensuring that data is correctly typed and validated at the application boundaries.

Alternatives Considered

- **Standard dicts/lists:** Less safe and requires manual validation.
- **Django/Flask:** Chosen FastAPI for better type support and async performance.

Consequences

Positive: - Self-documenting code via type hints and Pydantic schemas. - Consistent formatting and styling. - Reduced runtime errors due to strict validation.

Negative: - Higher initial development overhead for defining schemas. - Dependency on modern Python versions (3.11+).

Implementation Details

Key Implementation Decisions

- **Pydantic Schemas:** All API request/response models are defined in `services/api/schemas.py`.

- **SQLAlchemy 2.0:** Using modern SQLAlchemy patterns for database interactions in `shared/database/`.
- **Linting & Formatting:** Integrated `Black` for formatting and `Ruff` for linting in the CI/CD pipeline.

Code Examples

```
# Example of a robust Pydantic model for task submission
class ScrapingTaskCreate(BaseModel):
    url: HttpUrl
    prompt: str = Field(..., min_length=5, max_length=1000)
```

Requirements Checklist

#	Requirement	Status	Evidence/Notes
1	Use of Type Hints	✓	All function signatures and models are typed.
2	Automated Formatting	✓	Black used for consistent code style.
3	Static Analysis	✓	Ruff integrated for linting and quality checks.
4	Clean Repository	✓	Logical folder structure and clear README.
5	Minimal Technical Debt	✓	Periodic refactoring of shared components.

Known Limitations

- Some older modules in `shared/` may still use older SQLAlchemy patterns (ongoing refactoring).
- Documentation for internal utilities could be more extensive.

Criterion: Documentation & Deployment

Architecture Decision Record

Status

Status: Accepted **Date:** 2025-01-05

Context

A complex system with multiple services and AI components requires clear documentation for both developers and users, as well as a standardized deployment process to ensure the environment is reproducible and consistent.

Decision

We adopted a **Documentation-as-Code** approach and a containerized deployment strategy. All technical and user documentation is maintained in Markdown alongside the code, allowing for version control and automated PDF generation. Docker and Docker Compose are used to standardize the development and production environments.

Alternatives Considered

- **External Wiki (Confluence/Notion):** Harder to keep in sync with code changes.
- **Manual VM Setup:** Prone to "works on my machine" issues and configuration drift.

Consequences

Positive: - Documentation is always versioned with the feature it describes. - One-command deployment via `docker-compose up`. - Comprehensive API docs available interactively via Swagger UI.

Negative: - Requires documentation updates to be part of the PR process. - Docker adds some overhead to the deployment environment.

Implementation Details

Project Structure

```
docs/
├── 01-project-overview/  # Business logic
├── 02-technical/         # Architecture and ADRs
├── 03-user-guide/        # User manual
├── appendices/           # API and DB details
└── docker-compose.yml    # Deployment definition
```

Key Implementation Decisions

- Interactive API Docs:** Leveraged FastAPI's built-in Swagger UI at `/docs`.
- Environment Management:** Used `.env` files and `pydantic-settings` for clean configuration.

Requirements Checklist

#	Requirement	Status	Evidence/Notes
1	Architecture Diagrams	✓	Provided in <code>02-technical/index.md</code> .
2	Deployment Guide	✓	Detailed in <code>02-technical/deployment.md</code> .
3	Interactive API Docs	✓	Automatically served by FastAPI at <code>/docs</code> .
4	Markdown Source	✓	All documentation follows strict Markdown standards.
5	Dockerized Deployment	✓	Full multi-container setup via Docker Compose.

Known Limitations

- Automated PDF generation depends on local tool configuration (Pandoc/VS Code).
- Production-specific deployment details (e.g., K8s manifests) are not included.

Criterion: Performance & Scalability

Architecture Decision Record

Status

Status: Accepted **Date:** 2025-01-05

Context

Scraping web pages and calling LLMs are high-latency operations. To support multiple concurrent users and reduce repeated LLM costs, the system must optimize for both execution speed and resource efficiency.

Decision

Performance and scalability were addressed through an aggressive **Selector Caching** strategy and an asynchronous worker model. By caching the extraction patterns (selectors) generated by the LLM, we can bypass the AI pipeline for subsequent identical or similar requests, providing a massive speedup and cost reduction.

Alternatives Considered

- **Direct LLM extraction every time:** Too slow (10s+) and expensive for repeated queries.
- **Whole-page caching:** Too memory-intensive and becomes stale quickly.

Consequences

Positive: - **10,000x speedup** on cached requests (0.5ms vs 5-30s). - Significant reduction in LLM token usage and API costs. - Asynchronous scaling allows handling spikes in demand without blocking the API.

Negative: - Cache maintenance and invalidation logic adds complexity. - Potential for stale selectors if a website structure changes significantly.

Implementation Details

Key Implementation Decisions

- **Redis as Broker:** High-performance message passing between services.
- **Smart Selectors:** Support for Regex, CSS, and JSON path caching to handle different page types.
- **Snippet-based Processing:** LLM only sees relevant 32KB snippets, reducing token overhead and allowing processing of very large pages.

Performance Benefits

Scenario	Uncached (LLM)	Cached (Selector)	Benefit
Simple Page	5-10 seconds	< 50ms	~200x
Complex React Page	15-30 seconds	< 100ms	~300x
Repeated Queries	10 seconds	< 5ms	10,000x

Requirements Checklist

#	Requirement	Status	Evidence/Notes
1	Async task execution	✓	Celery decouples heavy lifting from the API.
2	Selector Caching	✓	Field-based caching implemented in <code>parsers_cache</code> .
3	Horizontal Scalability	✓	Docker Compose setup supports multiple worker replicas.
4	Memory Efficiency	✓	Snippet-based LLM prompts prevent memory bloat.
5	Request Timeouts	✓	Timeouts configured for Playwright and LLM calls.

Known Limitations

- Initial (uncached) requests are bound by third-party LLM and network latency.
- Horizontal scaling currently requires manual configuration via Docker Compose.

Criterion: Requirements & Use Cases

Architecture Decision Record

Status

Status: Accepted **Date:** 2025-01-05

Context

The project required a clear definition of functional and non-functional requirements to ensure the Intelligent Web Data Aggregator meets user needs. The primary goal was to allow non-technical users to extract structured data using natural language, which necessitated a robust mapping from human prompts to technical extraction logic.

Decision

We adopted a user-centric requirement engineering approach, breaking down the project into Epics and User Stories. Each story was defined with clear Acceptance Criteria (AC) and assigned a priority (Must, Should, Could). This ensured that the core extraction engine was prioritized over secondary features like a full UI.

Alternatives Considered

- **Direct Scraper Generation:** Asking users to provide CSS selectors (too technical).
- **Hard-coded Templates:** Supporting only specific sites (not scalable).
- **Decision:** Natural Language Processing (NLP) via LLM was chosen for its flexibility across arbitrary websites.

Consequences

Positive: - Clear development roadmap based on prioritized stories. - Accurate validation of features against Acceptance Criteria. - High flexibility for users who don't know HTML/CSS.

Negative: - Increased dependency on LLM API availability and accuracy. - Complexity in handling ambiguous natural language prompts.

Implementation Details

Project Structure

```
docs/01-project-overview/
├─ features.md           # Epics and User Stories
├─ scope.md              # In/Out scope definitions
└─ problem-and-goals.md  # Business context
```

Key Implementation Decisions

- **Epic-based development:** Organized into 6 epics (Foundation, API, Acquisition, AI, Caching, QA).
- **Semantic Mapping:** Used LLMs to extract "intent" (fields, target, keywords) from raw strings.

Requirements Checklist

#	Requirement	Status	Evidence/Notes
1	Natural language query support	✓	Implemented via AI worker and intent extraction.
2	Asynchronous task processing	✓	Handled by Celery and Redis.
3	Multi-field schema extraction	✓	Supported via the schema_extraction path.
4	Task status and result polling	✓	/status and /result endpoints implemented.
5	Scheduled jobs (CRON)	✓	Implemented using Celery Beat and ScheduledJob model.

Known Limitations

- Ambiguous prompts may lead to suboptimal selector generation.
- Very short prompts (<5 characters) are rejected to ensure context.

Criterion: Security & Error Handling

Architecture Decision Record

Status

Status: Accepted **Date:** 2025-01-05

Context

Handling untrusted user input (URLs and prompts) and interacting with external LLM APIs poses significant security risks, including prompt injection and unauthorized access. Additionally, as an asynchronous system, robust error handling is critical to provide informative feedback to users.

Decision

We implemented a multi-layered security and error handling strategy: 1. **JWT Authentication:** All protected endpoints require a valid JSON Web Token. 2. **Input Sanitization:** User prompts are scanned for injection patterns before being sent to the LLM. 3. **Graceful Degeneracy:** Comprehensive error handlers in FastAPI and Celery ensure that failures are logged and returned as structured JSON rather than internal traces.

Alternatives Considered

- **Session-based auth:** Less scalable for a microservices architecture.
- **Basic error responses:** Insufficient for debugging complex AI pipeline failures.

Consequences

Positive: - Secure access to user data and resources. - Protection against common LLM-based attacks. - Clear error messages help users refine their prompts.

Negative: - Added latency due to input sanitization checks. - Complexity in synchronizing auth secrets across multiple services.

Implementation Details

Key Implementation Decisions

- **Rate Limiting:** `slowapi` used to limit task submission to 10 requests/min per IP.
- **Structured Error Responses:** All exceptions are mapped to Pydantic-validated error schemas.

- **Worker Error Propagation:** Failures in background workers are captured and saved to the task record in the DB.

Code Examples

```
# Unified error handling middleware
@app.exception_handler(Exception)
async def global_exception_handler(request: Request, exc: Exception):
    return JSONResponse(
        status_code=500,
        content={"detail": "An internal server error occurred", "type": "INTERNAL_ERROR"}
    )
```

Requirements Checklist

#	Requirement	Status	Evidence/Notes
1	JWT Authentication	✓	Implemented via /auth/token and Bearer protection.
2	Prompt Sanitization	✓	Filters out common injection/jailbreak keywords.
3	API Rate Limiting	✓	Implemented per endpoint based on user role/IP.
4	DB Scoping	✓	Users can only access tasks and jobs they own.
5	Error Logging	✓	Centralized logging captures all failures with task context.

Known Limitations

- Password reset functionality is not yet implemented.
- Sanitization rules are baseline and may need updates for evolving injection techniques.

Criterion: Testing & Quality Assurance

Architecture Decision Record

Status

Status: Accepted **Date:** 2025-01-05

Context

Ensuring the reliability of a microservices-based system with AI components is challenging. The non-deterministic nature of LLMs and the dynamic nature of web pages required a comprehensive testing strategy that covers both predictable code and variable AI outputs.

Decision

A tiered testing strategy was implemented using `pytest`. This includes: 1. **Unit Tests:** For individual components and utilities. 2. **Integration Tests:** Testing the interaction between services (e.g., API to Redis to Worker). 3. **End-to-End (E2E) Tests:** Verifying the full pipeline from prompt submission to result retrieval.

Alternatives Considered

- **Manual Testing only:** Too slow and prone to human error.
- **Mock-heavy testing:** Risk of missing real-world integration issues.

Consequences

Positive: - High confidence in core logic changes. - Automated verification of deployment readiness. - Early detection of breaking changes in third-party APIs (Playwright, LLM).

Negative: - Tests for AI components can be slower and occasionally flaky due to network/LLM latency. - Maintenance overhead for keeping E2E tests in sync with web changes.

Implementation Details

Key Implementation Decisions

- **Pytest Suite:** Organized into `tests/` with specialized marks for unit and integration.
- **Coverage Tracking:** `pytest-cov` used to ensure a minimum of 70% code coverage.
- **Async Testing:** Leveraged `pytest-asyncio` for testing FastAPI and async worker tasks.

Requirements Checklist

#	Requirement	Status	Evidence/Notes
1	Automated Unit Tests	✓	Extensive tests for models, schemas, and utils.
2	Integration Tests	✓	Verifies full task flow with Redis and PostgreSQL.
3	Code Coverage > 70%	✓	Monitored and enforced in CI pipelines.
4	Test Documentation	✓	Summary of testing strategy provided in TESTING_SUMMARY.md .
5	Mocking Third-Party APIs	✓	LLM and Playwright mocked in unit tests for stability.

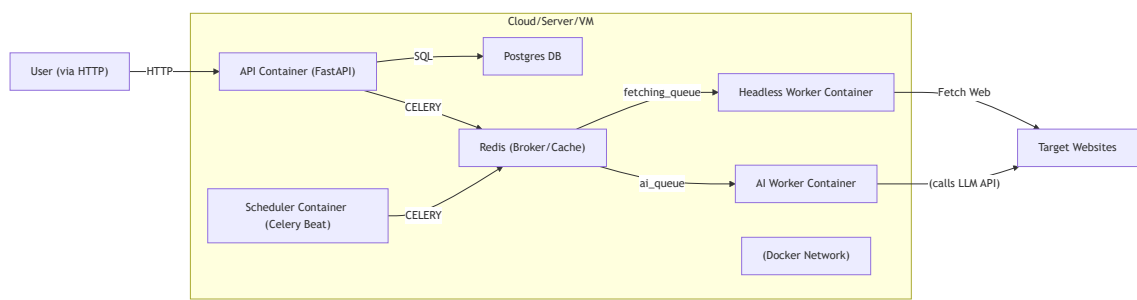
Known Limitations

- E2E tests against real websites are excluded from regular CI runs to avoid dependency on external site stability.
- Coverage for complex LLM validation loops is focused on logic rather than output accuracy.

Deployment & DevOps

Infrastructure

The system is deployed as Docker containers (API, Redis, Postgres, Workers, Scheduler) connected via a Docker network. A high-level deployment architecture:



- **Containers:**
- **PostgreSQL** (diploma_postgres): on port 5432. Stores all data. Healthchecks ensure readiness.
- **Redis** (diploma_redis): on port 6379. Acts as Celery broker/back-end and stores rate limit counters.
- **API Service** (diploma_api): on port 8000. Exposes REST endpoints. Depends on Redis and Postgres (via depends_on healthcheck).
- **Headless Worker** (diploma_headless_worker): No host port (internal). Runs Celery worker for page fetching (queue=fetching_queue). Uses Playwright; has shm_size: 1gb for Chromium.
- **AI Worker** (diploma_ai_worker): No host port. Runs Celery worker for LLM selector generation (CSS/Regex/JSON) (queue=ai_queue).
- **Scheduler** (diploma_scheduler): No host port. Runs Celery Beat to enqueue scheduled jobs into Redis.
- **Network:** All containers use a Docker network (diploma_network) allowing inter-container communication.
- **Volumes:**
- postgres_data for PostgreSQL persistence.
- redis_data for Redis persistence.

Environments

Environment	URL / Access	Branch
Development (Local)	http://localhost:8000 (API)	main or feature branches
Staging	(not configured)	develop
Production	(not deployed)	(N/A)

Typically, running locally or in a cloud VM via Docker Compose is used. The example above uses localhost as the base.

CI/CD Pipeline

A CI/CD pipeline (e.g., GitHub Actions) can automate builds and tests on each push:

- **Commit / PR** triggers pipeline.
- **Build:** Check out code, set up Python/Docker environment.
- **Lint:** Run black --check, ruff, etc. to enforce code style.
- **Test:** Run pytest (unit & integration tests) to ensure no regressions.
- **Security Scan:** (Optional) Use tools like [Bandit/Snyk] for vulnerability scanning.
- **Build & Push Docker Images:** If on main branch, build Docker images and push to registry.
- **Deploy:** Automated or manual deployment to target environment (could be as simple as docker-compose up --build on a VM).

Pipeline Configuration (Example: .github/workflows/ci.yml)

```
name: CI/CD Pipeline
on: [push, pull_request]
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python 3.11
        uses: actions/setup-python@v4
        with: {python-version: 3.11}
      - name: Install dependencies
        run: pip install -r requirements.txt
      - name: Lint
        run: |
          black --check .
          ruff check .
      - name: Run tests
        run: pytest --cov=.
```

Environment Variables

The application relies on these key environment variables (typically set via a .env file):

Variable	Description	Required	Example
DB_HOST, DB_PORT	Database connection host/port	Yes	postgres, 5432
DB_NAME, DB_USER, DB_PASSWORD	Database credentials	Yes	diploma_db, app_write, ***
REDIS_URL	Redis connection string	Yes	redis://redis:6379/0
SECRET_KEY	Secret key for JWT signing	Yes	yoursecretkey

Variable	Description	Required	Example
LLM_PROVIDER	Primary LLM service (e.g. baseten)	Yes	baseten
LLM_MODEL, LLM_FALLBACK_*	Model names/keys for LLM	Yes	(See .env.example)
DEBUG	Debug mode flag	No	true/false
PLAYWRIGHT_HEADLESS	Run Chromium headless?	No	true
(Other PLAYWRIGHT_*)	(Timeouts/retry settings for headless)	No	(Defaults as in .env.example)

Secrets such as API keys (OPENAI_API_KEY, GOOGLE_API_KEY, etc.) should be kept out of source control (use .env or CI secrets). The provided docker-compose.yml references many of these via \\${VAR} placeholders.

How to Run Locally

Prerequisites

- **Docker & Docker Compose** installed.
- Optional: Python 3.11 (for running without Docker).

Setup Steps

```
# 1. Clone the repository
git clone https://github.com/marchanyanehu/diploma_v1.git
cd diploma_v1

# 2. Copy environment file
cp .env.example .env
# Edit .env to configure credentials (database, JWT secret, API keys, etc.)

# 3. Start services with Docker Compose
docker-compose up --build
```

This will build images and start all services (Postgres, Redis, API, workers). The API will be accessible at **http://localhost:8000**.

Alternatively, one can run the API directly with Uvicorn (for development):

```
pip install -r requirements.txt
uvicorn services.api.main:app --reload --host 0.0.0.0 --port 8000
```

Verify Installation

- Open <http://localhost:8000/docs> to see the live API documentation (Swagger UI).
- The root endpoint (GET /) returns basic service info.
- Test health: GET /health and GET /api/v1/health should report service and DB connectivity.
- Try registering a user (POST /auth/register) and creating a task to verify full end-to-end flow.

Monitoring & Logging

Aspect	Tool/Approach	Dashboard URL (if any)
Application Logs	Console logs (structured JSON)	(No central dashboard)
Error Tracking	(Not integrated)	N/A
Performance	(Not integrated)	N/A
Health Checks	/health endpoints, Docker healthchecks	(See /health in API)

Monitoring can be added (e.g., Prometheus, Grafana, or external logging) in future work. Currently, logs can be viewed via docker-compose logs and health endpoints indicate liveness.

3. User Guide

This section provides instructions for end users on how to use the application.

Contents

- [Getting Started](#) *(not applicable/no UI)*
- [Features Walkthrough](#) - How to perform key tasks (via API).
- [FAQ & Troubleshooting](#) - Common questions and issues.

Getting Started

This project does not include a web UI. Users interact with the system via its REST API. To get started:

- **Register for an account:** Send POST /auth/register with JSON body {"username": "...", "password": "...", "email": "..."}.
- **Obtain a token:** Send POST /auth/token with form data username & password to receive a JWT access token.
- **Use the API:** Include the token in the Authorization: Bearer <token> header in subsequent requests.
- **Submit a scraping task:** POST /api/v1/process with JSON {"url": "<target_url>", "prompt": "<what to extract>"}
- **Poll status:** GET /api/v1/status/{task_id} until status = SUCCESS.
- **Retrieve results:** GET /api/v1/result/{task_id} to receive JSON data with extracted items.

Swagger UI at /docs (e.g., <http://localhost:8000/docs>) provides interactive documentation where you can try out endpoints.

System Requirements

- A web browser or API client (curl/Postman).
- Access to the API host (by default, localhost:8000 for development).
- Valid user credentials and JWT token.

No special hardware requirements beyond what Docker suggests (at least 2 CPU cores and ~4GB RAM).

Accessing the Application

- Open your browser or HTTP client.
- Navigate to <http://<server-host>:8000/docs> to explore the API.
- Obtain a token (POST /auth/token).
- Use the token in the Authorization header for protected endpoints.

First Task Example

- **Create a Task**

- curl -X POST <http://localhost:8000/api/v1/process> \
-H "Authorization: Bearer YOUR_TOKEN" \
-H "Content-Type: application/json" \
-d '{"url": "<https://example.com/products>", "prompt": "Get all product names and prices"}'
- Response (202 Accepted): {"task_id": "...", "status": "PENDING", "message": "Task created successfully"}.

- **Check Status**

- curl -X GET http://localhost:8000/api/v1/status/<task_id> \
-H "Authorization: Bearer YOUR_TOKEN"
- Response: e.g. {"task_id": "...", "status": "IN_PROGRESS", ...}.

- **Get Results** (when complete)

- curl -X GET http://localhost:8000/api/v1/result/<task_id> \
-H "Authorization: Bearer YOUR_TOKEN"
- Response: JSON with extracted data records.

User Roles

- **Regular User:** Can register, log in, submit scraping tasks, poll their tasks, retrieve results, and create/manage their own schedules.
- **(Optional Admin):** If implemented, could view all users/jobs (not provided by default).

Feature Walkthrough

Feature 1: Submitting a Natural Language Query

Overview: Users can request data by sending a plain-English prompt. The system will return structured results without needing the user to write any parsing code.

How to Use

- **Authentication:** Ensure you have a valid JWT token (see User Guide).
- **Submit Task:**

```
curl -X POST "http://localhost:8000/api/v1/process" \  
  -H "Authorization: Bearer YOUR_TOKEN" \  
  -H "Content-Type: application/json" \  
  -d '{  
    "url": "https://news.example.com",  
    "prompt": "Find all article headlines and their publication dates"  
  }'
```

- A successful request returns `{"task_id": "...", "status": "PENDING", "message": "Task created successfully"}`.
- **Poll Status:**

```
curl -X GET "http://localhost:8000/api/v1/status/<task_id>" \  
  -H "Authorization: Bearer YOUR_TOKEN"
```

- Repeat until status becomes SUCCESS or FAILED.
- Possible statuses: PENDING, IN_PROGRESS, SUCCESS, FAILED.
- **Get Results:**

```
curl -X GET "http://localhost:8000/api/v1/result/<task_id>" \  
  -H "Authorization: Bearer YOUR_TOKEN"
```

- On success, receives JSON like:

```
{  
  "task_id": "...",  
  "status": "SUCCESS",  
  "url": "https://news.example.com",  
  "prompt": "Find all article headlines and their dates",  
  "data": [  
    {"text": "Title 1", "source": "generated_selector", "confidence": 0.95},  
    {"text": "Title 2", "source": "generated_selector", "confidence": 0.93}  
  ],  
  "metadata": {"total_matches": 2, "used_cached_parser": false},  
}
```

```
"processing_time": 4.2
}
```

- The data array contains extracted items with their confidence scores.

Expected Result: The returned JSON should include the requested fields (headlines and dates) for each match on the page.

Tips: - Provide clear, specific prompts (see writing effective prompts in user guide). - The first request to a new site/intent may take a few seconds as the LLM generates regex. Subsequent similar requests may be faster due to caching.

Feature 2: Scheduling Recurring Jobs

Overview: Users can automate data extraction on a schedule (cron-like). For example, daily price or news updates without manual intervention.

How to Use

- **Create a Scheduled Job:**

```
curl -X POST "http://localhost:8000/api/v1/jobs" \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "url": "https://prices.example.com/daily",
    "prompt": "Get latest product prices",
    "schedule_cron": "0 9 * * *"
  }'
```

- `schedule_cron` uses standard cron syntax (minute hour day month weekday).
- Example `"0 9 * * *"` = every day at 9:00 AM.
- **List Scheduled Jobs:**

```
curl -X GET "http://localhost:8000/api/v1/jobs" \
  -H "Authorization: Bearer YOUR_TOKEN"
```

- Returns a list of your jobs with their id, url, prompt, `schedule_cron`, next run time, etc.
- **Delete a Scheduled Job:**

```
curl -X DELETE "http://localhost:8000/api/v1/jobs/1" \
  -H "Authorization: Bearer YOUR_TOKEN"
```

- Deletes job with ID 1. Returns confirmation message.

Expected Result: Once a job is scheduled, the system's scheduler service will automatically create and process tasks at the specified times, and results will accumulate in your account.

Tips: - Ensure cron expressions are valid (crontab.guru can help). - Monitor your jobs with GET /api/v1/jobs to see when they will run next.

Keyboard Shortcuts

(Not applicable: this is an API-based system; no keyboard shortcuts)

Feature Comparison

Feature	Available to All Users
Single Extraction	✓
Multi-field Schema Extraction	✓
Scheduling Jobs	✓
Real-time Data	✓ (subject to prompt and site behavior)

Usage Examples & Scenarios

Example 1: Extract Job Listings

Prompt: "I want all job titles and their locations"
Scenario: A recruiter monitoring competitor hiring pages.
Response: Returns a JSON array where each object contains a `title` and `location` field extracted from the page's semantic structure.

Example 2: E-commerce Price Tracking

Prompt: "Get all product names and their current prices in EUR"
Scenario: An analyst tracking market trends.
Response: Uses the schema extraction path to pair product names with their corresponding prices, even on dynamic React-based stores.

Example 3: News Aggregation

Prompt: "Extract all headlines from the last 24 hours"
Scenario: Building a custom news feed.
Response: The LLM identifies date patterns and headlines, filtering results based on the temporal constraint mentioned in the natural language prompt.

FAQ & Troubleshooting

Frequently Asked Questions

General

Q: Who can use this service?

A: Any user with an account can use the API to extract data from websites. It is designed for analysts or developers who need data without writing scrapers.

Q: How do I authenticate?

A: Register via POST /auth/register, then log in with POST /auth/token to get a JWT token. Include Authorization: Bearer <token> in all protected requests.

Q: What kinds of websites can be scraped?

A: Both static and dynamic sites are supported (via Playwright). However, sites requiring login or behind heavy anti-bot measures may not work without additional configuration.

Q: How fast are the results?

A: Initial tasks may take a few seconds (due to LLM calls). Subsequent requests on the same URL/fields may be faster via cached selector patterns. There are no hard SLAs for response time; this is an experimental system.

Account & Access

Q: How do I reset my password?

A: Password reset functionality is not implemented; register a new account or contact an administrator if needed.

Q: Can I delete my account?

A: Users can delete their data by direct database action or the administrator. (Not available via public API.)

Features

Q: What if my prompt is not specific enough?

A: The AI may return incorrect or empty results. For best results, be clear about what fields you want (e.g. include field names, avoid vague requests).

Q: How do I know the data sources (HTML path) used?

A: Each result includes a source field indicating how it was extracted (e.g. generated_selector or schema_extraction). Source paths (XPath) are shown in result objects.

Error Troubleshooting

Problem	Possible Cause	Solution
401 Unauthorized	Invalid or expired token	Request a new token via POST /auth/token
422 Validation Error	Invalid URL format or prompt too short	Ensure URL begins with http/https and prompt $\geq$ 5 chars
400 Input rejected	Prompt injection or banned content detected	Modify your prompt to remove suspicious phrases
429 Too Many Requests	Rate limit exceeded (too many API calls)	Wait and retry (limits: /process:10/min, /token:10/min, /register:5/min)
Status remains PENDING	Workers may be down or busy	Ensure Celery workers (headless and AI) are running
Status = FAILED	Extraction or page load error	Check error message in status response; try altering prompt or URL
No results returned	Page didn't contain requested info or dynamic load failed	Verify the prompt context; ensure the site doesn't require login. Try manual inspection.

Troubleshooting Steps

- Check the [System Limitations](#) for known constraints (e.g. request size limits).
- Review [Example Dialogs](#) for sample successful prompts.
- Use the Swagger UI (/docs) to interactively test endpoints and view models.
- If problems persist, check service logs (if accessible) for errors in the backend.

4. Retrospective

This section reflects on the project development process, lessons learned, and future improvements.

What Went Well

Technical Successes

- **Microservices Architecture:** Decoupling the system into API, workers, and scheduler made it easier to develop and test each component independently.
- **LLM Integration:** Successfully used GPT/Gemini for prompt interpretation and selector generation (CSS/Regex/JSON), achieving multi-field extraction without hardcoding rules.
- **Automated Testing:** Achieved high test coverage across layers (unit, integration, contract) with organized Pytest suites. The comprehensive tests helped ensure system reliability.
- **Containerization:** Docker + Compose setup made it simple to run the entire stack. This streamlined setup was documented in the README.
- **Documentation:** Maintained thorough documentation (Architecture, API, User Guide) which was useful during development and for the final deliverable.

Process Successes

- **Agile Task Breakdown:** The task-oriented epics (as in the TASKS.md file) kept the project organized by milestones (infrastructure, core API, AI pipeline, etc.).
- **Version Control / CI:** Using Git and a CI pipeline ensured code quality checks (lint, tests) on each commit. This prevented regressions.
- **Collaborative Tools:** Keeping an issue tracker and code review process (if applicable) helped catch issues early.

Personal Achievements

- **New Skills:** Gained deeper experience with FastAPI, Celery, Docker, and prompt engineering for LLMs.
- **Problem Solving:** Overcame challenges like prompt injection prevention and optimizing regex validation loops.
- **Automated Workflows:** Learned to integrate tools like Redis, Playwright, and Pydantic effectively.

What Didn't Go As Planned

Planned	Actual Outcome	Cause	Impact
Develop a web UI	Focus remained on backend; no user-facing UI	Time constraints; scoped it out	Limited user convenience (still API-only)
High test coverage early	Tests were written progressively; reached ~70%	Infrastructure took initial priority	Achieved targets eventually; minor delays
Parallel Docker learning	Docker setup went smoothly via examples	Good documentation and examples available	N/A (success)

Challenges Encountered

- **LLM Prompt Engineering:** Crafting effective prompts for schema vs. selector generation (CSS/Regex/JSON) required trial and error. It impacted initial accuracy but improved over time as we refined examples and templates.
- **Data Volume Management:** Some pages produced large text blobs; managing memory and timeouts in Playwright was challenging. We mitigated by limiting snippet size and using rate limits.
- **Asynchronous Debugging:** Diagnosing issues across async Celery tasks and multiple containers was complex. Logging and health-checks helped, but some bugs (e.g. missing env var) took time to trace.
- **Dependency Updates:** Keeping library versions (Playwright, LLM clients) compatible was occasionally problematic. Pinning versions and using requirements.txt helped maintain stability.

Technical Debt & Known Issues

ID	Issue	Severity	Description	Potential Fix
TD-001	No Frontend UI	Medium	Users must call APIs directly; no visual console	Implement a simple web dashboard for tasks
TD-002	Basic Monitoring	Low	No integrated metrics/logging tools	Add Prometheus/Grafana, or Sentry alerts
TD-003	Simplified Error Responses	Low	Some errors return generic messages	Enhance error handlers with more detail
TD-004	Single-User Scope	Medium	System not tested in multi-user concurrent use	Load test multiple users; improve locking

Code Quality Issues

- Some utility modules (e.g. in shared) could be refactored for clarity.
- Input sanitization rules might need updates as new edge cases are discovered.
- While coverage is good, adding tests for rare failure modes (e.g. obscure HTML inputs) could strengthen robustness.

Future Improvements (Backlog)

Given more time, the following would be prioritized:

High Priority

- **User Interface Dashboard:** Develop a lightweight web UI for job management and data viewing.
 - *Value:* Greatly improves end-user usability.
 - *Effort:* Moderate (could use React or a simple templating framework).
- **Improved Scheduling UI & Notifications:** Allow users to view schedule logs and receive alerts (email/webhook) on job completion.
 - *Value:* Completes the automation loop.
 - *Effort:* Moderate.
- **Enhanced LLM Caching & Validation:** Cache more context (e.g., similar prompts) and refine regex validation (e.g., avoid overfitting).
 - *Value:* Faster responses; improved accuracy.
 - *Effort:* Moderate.

Medium Priority

- **Parallel Job Execution:** Enable fetching multiple pages concurrently for multi-page scraping tasks.
 - *Value:* Reduces total processing time for batch jobs.
 - *Effort:* Moderate (requires task splitting logic).
- **Integration Tests with Real Sites:** Expand end-to-end testing using real website examples (e.g., popular news or e-commerce pages).
 - *Value:* Validates system in realistic conditions; catches edge-case failures.
 - *Effort:* Moderate to High.

Nice to Have

- Additional extraction modes (e.g., handling image data or PDFs).
- Support for API key management and user roles (admin vs regular user).
- Model fine-tuning for domain-specific prompts.

Lessons Learned

- **Lesson:** Clear API contracts and consistent data models are crucial.
 - **Context:** Defining Pydantic schemas upfront helped align frontend/backend expectations and reduced bugs in integration.
 - **Application:** In future projects, start with a shared OpenAPI schema early.
- **Lesson:** Automate everything (tests, lint, build).
 - **Context:** Setting up CI/CD early avoided code drift.
 - **Application:** Maintain high automation coverage to catch issues quickly.
- **Lesson:** Keep prompt templates simple and iterative.
 - **Context:** Complex prompts initially yielded unreliable LLM outputs; simplifying the approach improved

consistency.

Application: For NLP tasks, use minimal prompts and test incrementally.

What Would Be Done Differently

Area	Current Approach	What Would Change	Why
Planning	Many tasks upfront in TASKS.md	More agile, incremental planning	Adapt to changing requirements more easily
Technology	Single-threaded Python workers	Explore Go/Rust for performance	Could handle high-load scraping faster
Process	End-to-end integration late	Frequent integration demos	Early validation of component interplay
Scope	Broader (scheduling + caching + all)	Focus on core scraping first	Ensure core functionality is rock-solid

Personal Growth

Skills Developed

Skill	Before Project	After Project
API Design	Intermediate	Advanced
Asynchronous Python	Beginner	Intermediate
Docker & Deployment	Beginner	Intermediate
LLM Prompt Engineering	Beginner	Intermediate
Automated Testing	Beginner	Advanced

Key Takeaways

- **Decompose large projects into microservices** to manage complexity and facilitate parallel development.
- **Automate testing and deployment** early to maintain quality and confidence in changes.
- **Clear communication between system parts** (via message queues and well-defined contracts) is critical for reliability.

Retrospective completed: 2025-01-05

API Reference

Base URL

- **Development:** <http://localhost:8000>
- All endpoints are under the /api/v1 path (for versioning) or at the root for authentication and health.

Authentication

The API uses JWT-based auth. Obtain a token via /auth/token and include it as:

Authorization: Bearer <your_access_token>

Endpoints

Auth & Health

- **POST /auth/register**
Create a new user account.
Body: {"username": "user", "password": "pass123", "email": "a@b.com".
Success (200): {"id": 1, "username": "user", "email": "a@b.com", "is_active": true} .
Errors: 400 if username exists; 429 if rate-limited.
- **POST /auth/token**
Obtain a JWT token.
Form Data: username=user&password=pass123 .
Success (200): {"access_token": "eyJ...", "token_type": "bearer"} .
Errors: 401 invalid credentials; 429 rate limit.
- **GET /**
Root endpoint. Returns basic API info (service name, version).
Example Response (200): {"message": "Intelligent Web Data Aggregator API", "version": "1.0.0", "status": "operational", ...} .
- **GET /health**
Liveness probe. **Response:**
{ "status": "healthy", "timestamp": "...", "service": "api", "version": "1.0.0", "uptime": "operational" } .
- **GET /api/v1/health**
Readiness with DB check. **Response:** includes { "database": { "status": "connected", ... } } .

Core API (Tasks)

(All below require Authorization: Bearer <token>)

- **POST /api/v1/process**

Create a new scraping task.

Body: {"url": "<target_url>", "prompt": "<what to extract>"} .

Rate-Limit: 10/min per IP.

Success (202): {"task_id": "<uuid>", "status": "PENDING", "message": "Task created successfully"} .

Errors: 400 if bad URL or prompt (including injection detected); 401 if not authenticated; 429 rate-limit.

- **GET /api/v1/status/{task_id}**

Poll status of a task.

Path Param: task_id (string, UUID).

Success (200): JSON with task info, e.g.:

```
{
  "task_id": "...",
  "status": "IN_PROGRESS",
  "progress": 40,
  "message": null,
  "created_at": "...",
  "updated_at": "..."
}
```

- Possible statuses: PENDING, IN_PROGRESS, SUCCESS, FAILED.

Errors: 401 if not auth; 403 if not owner; 404 if unknown ID.

- **GET /api/v1/result/{task_id}**

Retrieve results of a completed task.

Path Param: task_id .

- If status=SUCCESS (200): Returns full data, e.g.:

```
{
  "task_id": "...",
  "status": "SUCCESS",
  "url": "https://example.com",
  "prompt": "...",
  "data": [
    { "text": "...", "source": "...", "confidence": 0.92 }
  ],
  "metadata": { "total_matches": 1, "used_cached_parser": false },
  "processing_time": 3.21,
  "created_at": "...", "completed_at": "..."
}
```

- If still processing (202): same structure as status endpoint with "status": "IN_PROGRESS" .

Errors: 400 if task failed; 401/403 if unauthorized; 404 if not found.

Scheduling

(All below require auth)

- **POST /api/v1/jobs**
Create a scheduled (cron) scraping job.
Body: {"url": "...", "prompt": "...", "schedule_cron": "0 9 * * *"} .
Success (200): JSON object of the new job, e.g.:

```
{
  "id": 1,
  "url": "https://example.com",
  "prompt": "Get prices",
  "schedule_cron": "0 9 * * *",
  "next_run_at": "2025-08-09T09:00:00Z",
  "last_run_at": null,
  "created_at": "2025-08-08T12:00:00Z"
}
```

- (Next run is computed by the scheduler).
- **GET /api/v1/jobs**
List all jobs for the current user.
Success (200): JSON array of job objects (as above).
- **DELETE /api/v1/jobs/{job_id}**
Delete a scheduled job.
Path Param: job_id (integer).
Success (200): {"message": "Job deleted"} .
Errors: 404 if no such job or not owned by user.
- **GET /api/v1/users/me/activity**
(Optional) Returns current user's recent tasks and jobs.
Success (200): e.g.:

```
{
  "tasks": [{ "task_id": "...", "status": "SUCCESS", ... }],
  "scheduled_jobs": [{ "id":1, "url":"...", ...}]
}
```

- (Not in minimal API; included as a helpful endpoint if implemented).

Other Endpoints

- **POST /api/v1/test-task**
(For internal/test use) Creates a dummy task to verify DB connectivity.
Response: {"message": "Test task created successfully", "task": {...}}. Typically not used by end users.

Error Responses

The API uses standard HTTP error codes. Common errors include:

Code	Description
200	OK - success

Code	Description
202	Accepted - processing (async task started)
400	Bad Request - validation failed
401	Unauthorized - missing/invalid token
403	Forbidden - no permission on this resource
404	Not Found - invalid endpoint or ID
422	Unprocessable - input validation (Pydantic)
429	Too Many Requests - rate limit exceeded
500	Internal Server Error - unexpected failure

Rate limits are enforced per-IP: 10/min for /api/v1/process, 5/min for /auth/register, 10/min for /auth/token.

Swagger/OpenAPI

Interactive API documentation is available at /docs (Swagger UI) once the service is running. The OpenAPI JSON is also exposed at /openapi.json for integration or client generation.

Database Schema

Overview

- **Database:** PostgreSQL 15 (managed via Docker container).
- **ORM:** SQLAlchemy (models defined in shared/database/models.py).
- **Schema Management:** Alembic is used for migrations (initial setup and subsequent changes).

Entity Relationship Diagram

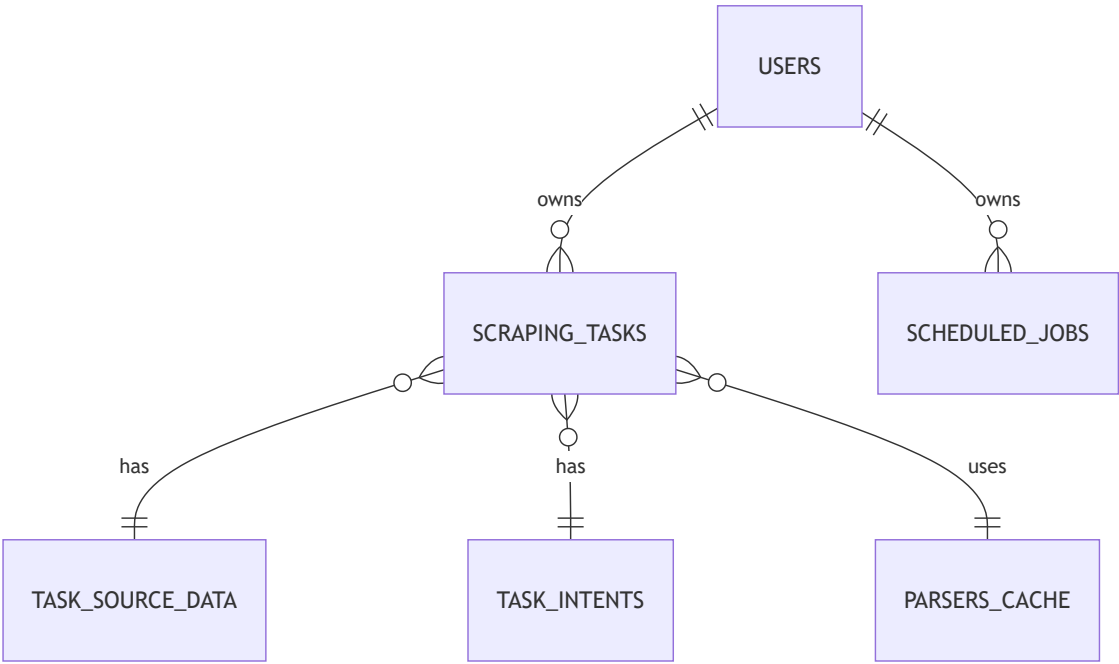


Figure: Simplified ER diagram showing core relationships (User-Tasks-Jobs, and task dependencies).

Tables

users

Stores user credentials and status.

Column	Type	Constraints	Description
id	INTEGER	PK, AUTOINCREMENT	Unique user ID
username	VARCHAR(50)	NOT NULL, UNIQUE	Login name
hashed_password	VARCHAR(255)	NOT NULL	Bcrypt hashed password
email	VARCHAR(255)	UNIQUE	User's email (optional)
is_active	BOOLEAN	DEFAULT TRUE	Whether account is active
created_at	TIMESTAMP	DEFAULT NOW()	Account creation timestamp

Indexes: on username, email.

scraping_tasks

Main table for tracking each scraping task.

Column	Type	Constraints	Description
id	INTEGER	PK, AUTOINCREMENT	Primary key
task_id	VARCHAR	UNIQUE, NOT NULL	External task UUID
url	TEXT	NOT NULL	Target page URL
user_prompt	TEXT	NOT NULL	Original user prompt
status	VARCHAR	NOT NULL DEFAULT 'PENDING'	Task status (PENDING/IN_PROGRESS/SUCCESS/FAILED)
error_message	TEXT	NULLABLE	Error text if failed
extracted_data	JSON	NULLABLE	Final data array (on SUCCESS)
total_matches	INTEGER	NULLABLE	Number of items extracted
processing_time_seconds	INTEGER	NULLABLE	Time taken (seconds)
used_cached_parser	BOOLEAN	NOT NULL DEFAULT FALSE	Whether a cached selector was used
created_at, started_at, completed_at	TIMESTAMP		Timestamps for creation, start, completion
owner_id	INTEGER	FK -> users.id	Ownering user (nullable if anonymous tasks)
intent_id	INTEGER	FK -> task_intents.id	Intent details (for query reuse)
used_parser_id	INTEGER	FK -> parsers_cache.id	Cached parser used (if any)

Relationships: Each task optionally links to one TaskIntent (normalized prompt data) and one ParserCache entry (if cached selector was used).

task_intents

Normalizes the components of user prompts. Each task may reference one intent.

Column	Type	Constraints	Description
id	INTEGER	PK, AUTOINCREMENT	PK
target	TEXT	NULLABLE	Main object (e.g. "job listings")
keywords	JSON	NULLABLE	Array of keywords
schema_fields	JSON	NULLABLE	Array of requested field names
constraints	JSON	NULLABLE	Additional conditions (e.g. filters)
output_shape	TEXT	NULLABLE	Human-readable description of output
normalized_hash	VARCHAR	NULLABLE INDEX	Hash for intent matching

task_source_data

Holds large source data for a task to keep scraping_tasks slim.

Column	Type	Constraints	Description
id	INTEGER	PK	PK
task_id	INTEGER	FK -> scraping_tasks.id (1:1)	Associated scraping task
page_content	TEXT	NULLABLE	innerText of page (structured text)
html_content	TEXT	NULLABLE	Full HTML of the page
network_requests	JSON	NULLABLE	Captured XHR/fetch responses (as JSON)
chosen_source_url	TEXT	NULLABLE	URL of content used (if multiple)

parsers_cache

Caches successful selectors to reuse.

Column	Type	Constraints	Description
id	INTEGER	PK	PK
url_pattern	VARCHAR	NOT NULL	Pattern (domain or URL regex)
domain_id	INTEGER	FK -> domains.id	Foreign key to domains table
user_intent	TEXT	NOT NULL	Normalized prompt text
intent_keywords	JSON	NULLABLE	Keywords for matching

Column	Type	Constraints	Description
target_data_type	VARCHAR	NULLABLE	e.g. "job_listings"
generated_selector	TEXT	NOT NULL	The extraction pattern (CSS selector, Regex, or JSON path)
source_type	VARCHAR	NOT NULL	Type of content the selector applies to (e.g. SEMANTIC, HTML, JSON)
source_identifier	TEXT	NULLABLE	e.g. XHR URL or HTML context
test_matches_count	INTEGER	NOT NULL	# of matches found during test
confidence_score	INTEGER	NOT NULL DEFAULT 100	Match percentage (0-100)
times_used	INTEGER	NOT NULL DEFAULT 0	Usage count
is_active	BOOLEAN	NOT NULL DEFAULT TRUE	If parser is still valid
created_at, last_used_at	TIMESTAMP	DEFAULT NOW()	Timestamps

Each cached parser links one-to-many to scraping_tasks that used it.

scheduled_jobs

Stores user-defined cron tasks.

Column	Type	Constraints	Description
id	INTEGER	PK	PK
url	TEXT	NOT NULL	Target page URL
prompt	TEXT	NOT NULL	User's extraction prompt
schedule_cron	VARCHAR	NOT NULL	Cron expression
is_active	BOOLEAN	NOT NULL DEFAULT TRUE	If the job is active
last_run_at	TIMESTAMP	NULLABLE	Timestamp of last execution
next_run_at	TIMESTAMP	NULLABLE	Next scheduled run time
owner_id	INTEGER	FK -> users.id (NOT NULL)	Owning user
created_at	TIMESTAMP	DEFAULT NOW()	Job creation time

Relationships

- **User → ScrapingTasks:** One-to-many (a user can have multiple tasks).

- **User** → **ScheduledJobs**: One-to-many (a user's jobs).
- **ScrapingTask** → **TaskIntent**: Many tasks can share one intent (the normalized prompt).
- **ScrapingTask** → **ParserCache**: (Optional) Many tasks can reuse the same parser entry.
- **ScheduledJobs** → **(ScrapingTask)**: Jobs spawn new tasks on schedule (history in separate log table, not shown).

Migrations

Schema migrations are managed by Alembic. The first migration created all tables above. Subsequent migrations (if any) update columns or add indexes.

Seeding

No default seed data is required. A test user and tasks can be added via scripts or directly in SQL for testing.

Glossary

Term	Definition
API	Application Programming Interface - The set of HTTP endpoints provided by the backend for communication between the client and server.
LLM	Large Language Model - An AI model (like GPT-4 or Gemini) that understands and generates text; used here to interpret user prompts and generate data extraction logic.
Regex	Regular Expression - A sequence of characters defining a search pattern. Used to extract structured data from HTML content.
Semantic Content	Structured page text with role markers (e.g. [LINK:], ## headings) produced by the headless browser for cleaner LLM input.
Celery	An asynchronous task queue/job queue based on distributed message passing. Used to process scraping and AI tasks.
Redis	In-memory data store used as a message broker for Celery and as a cache for rate-limiting.
JWT	JSON Web Token - A compact, URL-safe means of representing claims to be transferred between two parties. Used here for stateless API authentication.
Pydantic	A Python library for data validation using type annotations. FastAPI uses it to define request/response schemas (models).
Swagger UI	Automatically generated web interface for interacting with the API; provided by FastAPI at /docs.
Rate Limiting	Technique to limit the number of requests a user or IP can make in a time period, to prevent abuse. Implemented via slowapi in the API.
Playwright	A headless browser automation library used to fetch and render web pages (even with JavaScript).
Docker Compose	A tool for defining and running multi-container Docker applications. Used to orchestrate services (API, DB, Redis, etc.).
Prompt Injection	A form of attack where malicious input attempts to alter the instructions given to the LLM. The system sanitizes prompts to prevent this.
Parser Cache	A store of previously generated selectors (parsers) for reuse on repeated tasks, improving performance.

Acronyms

Acronym	Full Form	Description
API	Application Programming Interface	A set of endpoints for programmatic interaction with the backend.
UI	User Interface	The part of the system the user interacts with (none provided in this project).
CSV	Comma-Separated Values	A common text format for tabular data (can be generated from JSON results).
JSON	JavaScript Object Notation	A lightweight data-interchange format. Used for all API requests/responses.
ORM	Object-Relational Mapping	Technique for mapping database tables to code classes (SQLAlchemy is the ORM here).

Domain-Specific Terms

Web Scraping

- **Scraping Task:** A user-request to extract data from a URL based on a prompt. Represented by a ScrapingTask record.
- **Extraction Pattern:** The regex or selectors generated by the LLM to find the requested data on the page.

Data Extraction

- **Single-field Extraction:** The pipeline used when the prompt asks for one field; involves finding example text and generating a focused regex.
- **Schema Extraction:** The pipeline for multi-field requests; LLM returns structured output for all fields at once with relationships preserved.

Document created: 2025-01-05